- 古茗前端笔试
  - 简答题
    - 1. TAB组件实现
    - 2. 字符统计
    - 3. 合并区间
    - 4. 比较版本格式

古茗前端笔试

简答题

1. TAB组件实现

实现一个Tab组件，要求支出禁用，支持受控。

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</head>

<body>

<div class="demo-container">
    <div id="tab-root"></div>

    <!-- 受控演示：外部按钮控制激活选项卡 -->
    <div class="control-bar">
```

```
        <strong>受控演示（外部控制）:</strong><br/>
        <button id="btn-fruit">激活「水果」</button>
        <button id="btn-veg">激活「蔬菜」（禁用，无效）</button>
        <button id="btn-animal">激活「动物」</button>
    </div>
</div>

</body>

</html>

<script>
    /*
     * @param {HTMLElement} container - 挂载点
     * @param {Object} options - 配置项
     * @param {Array} options.items - Tab项 [{ key, label, content, disabled }]
     * @param {string} options.activeKey - 当前激活的 key（受控）
     * @param {Function} options.onChange - 切换回调 (key) => void
    */
    class Tabs {
        constructor(container, options) {
            this.container = container;
            this.items = options.items || [];
            this.activeKey = options.activeKey;
            this.onChange = options.onChange || (() => {});

            // 渲染 DOM
            this.render();
        }

        // 设置激活的 key
        setActiveKey(key) {
            if (this.activeKey === key) return; // 已经是当前激活的，不处理
            if (this.items.find(item => item.key === key)?.disabled) return; // 禁
用状态不处理
            this.activeKey = key;
            this.onChange(key); // 触发回调
            this._updateActiveUI(); // 更新 UI
        }

        // 更新按钮和内容的显示状态
        _updateActiveUI() {
            const buttons = this.container.querySelectorAll('.tab-btn');
            const panels = this.container.querySelectorAll('.tab-panel');

            // 更新按钮状态
            buttons.forEach(btn => {
                const btnKey = btn.getAttribute('data-key');
                if (btnKey === this.activeKey) {
                    btn.classList.add('active');
                } else {
                    btn.classList.remove('active');
                }
            });

            // 更新内容面板显示
            panels.forEach(panel => {
```

```
            const panelKey = panel.getAttribute('data-key');
            if (panelKey === this.activeKey) {
                panel.style.display = 'block';
            } else {
                panel.style.display = 'none';
            }
        });
    }

    // 处理选项卡点击
    _handleTabClick(key, disabled){
        if(disabled) return; // 禁用状态不处理
        this.setActiveKey(key);
    }

    // 全量渲染
    render() {
        if (!this.container) return;
        this.container.innerHTML = '';

        // 构建头部按钮区域
        const headerDiv = document.createElement('div');
        headerDiv.className = 'tabs-header';

        // 构建内容面板容器
        const contentDiv = document.createElement('div');
        contentDiv.className = 'tabs-content';

        this.items.forEach(element => {
            // 创建按钮
            const button = document.createElement('button');
            button.textContent = element.label;
            button.className = 'tab-btn';
            if (this.activeKey === element.key) button.classList.add('active');
            button.setAttribute('data-key', element.key);
            if (!element.disabled) {
                button.addEventListener('click', () =>
this._handleTabClick(element.key, element.disabled));
            } else {
                button.disabled = true; // 设置禁用状态
            }
            headerDiv.appendChild(button);

            // 创建内容面板
            const panel = document.createElement('div');
            panel.className = 'tab-panel';
            panel.setAttribute('data-key', element.key);
            panel.innerHTML = element.content;
            contentDiv.appendChild(panel);
        });

        this.container.appendChild(headerDiv);
        this.container.appendChild(contentDiv);

        this._updateActiveUI();
    }
}
```

```
    const tabItems = [
        { key: 'fruit', label: '水果', content: '水果列表：苹果、香蕉、橙子、葡萄。',
disabled: false },
        { key: 'veg', label: '蔬菜', content: '蔬菜列表：西兰花、胡萝卜、番茄、菠菜。',
disabled: true },
        { key: 'animal', label: '动物', content: '动物列表：熊猫、狮子、大象、长颈
鹿。', disabled: false }
    ];

    let currentActiveKey = 'fruit';   // 初始激活“水果”

    // 获取挂载容器
    const root = document.getElementById('tab-root');

    const myTabs = new Tabs(root, {
        items: tabItems,
        activeKey: currentActiveKey,
        onChange: (key) => {
            console.log('切换到:', key);
        }
    });

    // 外部按钮控制（演示受控）
    document.getElementById('btn-fruit').addEventListener('click', () => {
        myTabs.setActiveKey('fruit');
    });
    document.getElementById('btn-veg').addEventListener('click', () => {
        myTabs.setActiveKey('veg');
    });
    document.getElementById('btn-animal').addEventListener('click', () => {
        myTabs.setActiveKey('animal');
    });
</script>

<style>
    .tab-btn{
        height: 50px;
        width: 150px;
        font-size: large;
    }
    .tab-btn.active {
        background: #007bff;
        color: white;
    }
    .tab-btn.disabled {
        opacity: 0.5;
        cursor: not-allowed;
        background: #e9ecef;
    }
    .tab-panel{
        border: 1px solid #ccc;
        padding: 20px;
    }
    .control-bar{
        margin-top: 20px;
    }
```

```
    button{
        width: 200px;
    }
</style>
```

2. 字符统计

编写一个JS函数，传入一个非空字符串，计算出现最多的字符，返回该字符及出现次数，结果可能有多个。

```
输入：s = "xyzzyxyz"
输出：{y:3, z:3}
```

```
/**
 * @param {string} s
 * @return {Map}
 */
function frequencyChar(s){
    let max = 0;
    let res = new Map();
    let map = new Map();
    for(let i=0;i<s.length;i++){
        let c = s[i];
        if(map.has(c)){
            map.set(c,map.get(c)+1);
        }
        else{
            map.set(c,1);
        }
    }
    for(let [key,value] of map){
        if(value>max){
            max = value;
            res.clear();
            res.set(key,value);
        }
        else if(value === max){
            res.set(key,value);
        }
    }
    return res;
}

console.log(frequencyChar("hello world"));
console.log(frequencyChar("xyzzyxyz"));
```

3. 合并区间

以数组 `intervals` 表示若干个区间的集合，其中单个区间为 `intervals[i] = [start<sub>i</sub>, end<sub>i</sub>]` 。请你合并所有重叠的区间，并返回 *一个不重叠的区间数组，该数组需恰好覆盖输入中的所有区间*。(leetcode-56. 合并区间)

要求返回的区间数组按左端点升序排列，如果 `intervals`为空返回 `[]` 。

附加：给出代码的时间复杂度和空间复杂度。

```
输入：intervals = [[1,3],[2,6],[8,10],[15,18]]
输出：[[1,6],[8,10],[15,18]]
```

```
输入：intervals = [[1,4],[4,5]]
输出：[[1,5]]
```

```
输入：intervals = [[4,7],[1,4]]
输出：[[1,7]]
```

```
var merge = function(intervals) {
    let merge = [];
    intervals.sort((a,b) => a[0] - b[0]);   // 按区间起点升序排列，这样重叠的区间就会相邻
    for(let i=0;i<intervals.length;i++){
        let [start,end] = intervals[i];
        if(merge.length === 0 || merge[merge.length-1][1] < start){   // 如果当前区间与上一个区间不重叠，直接添加
            merge.push([start,end]);
        }
        else{   // 否则，更新上一个区间的结束位置
            merge[merge.length-1][1] = Math.max(merge[merge.length-1][1],end);
        }
    }
    return merge;
};

// 示例
intervals = [[1,3],[0,2],[2,6],[1,5],[8,10],[15,18]]
console.log(merge(intervals));
```

合并区间的核心思想是先排序，再合并： 排序：按区间起点升序排列，这样重叠的区间就会相邻 合并：遍历排序后的区间，若当前区间起点 ≤ 上一个区间的终点，则合并；否则加入结果

时间复杂度：O(nlogn)，其中 n 为区间的数量。除去排序的开销，我们只需要一次线性扫描，所以主要的时间开销是排序的 O(nlogn)。

空间复杂度：O(logn)，其中 n 为区间的数量。这里计算的是存储答案之外，使用的额外空间。O(logn) 即为排序所需要的空间复杂度。

4. 比较版本格式

给你两个 **版本号字符串** `version1` 和 `version2` ，请你比较它们。版本号由被点 `'.'` 分开的修订号组成。**修订号的值** 是它 **转换为整数** 并忽略前导零。比较版本号时，请按 **从左到右的顺序** 依次比较它们的修订号。如果其中一个版本字符串的修订号较少，则将缺失的修订号视为 `0`。（leetcode-165. 比较版本号）

返回规则如下：

- 如果 `version1 < version2` 返回 `-1`，
- 如果 `version1 > version2` 返回 `1`，
- 除此之外返回 `0`。

```
输入：version1 = "1.2.0", version2 = "1.10.0"
输出：-1
```

```
/**
 * @param {string} version1
 * @param {string} version2
 * @return {number}
 */
var compareVersion = function(version1, version2) {
    let v1 = version1.split('.').map(Number);
    let v2 = version2.split('.').map(Number);
    let len = Math.max(v1.length,v2.length);
    for(let i=0;i<len;i++){
        let num1 = i<v1.length?v1[i]:0;
        let num2 = i<v2.length?v2[i]:0;
        if(num1>num2){
            return 1;
        }
        else if(num1<num2){
            return -1;
        }
```

```
    }
    return 0;
};
```